Design Document - ULAgent-Sender

改訂履歴

版数	日付	改定内容	担当者
1.0	2026/07/22	初版作成	-

Overview

本ドキュメントは、ULAgentを構成する送信側コンポーネント「ULAgent-Sender」の技術設計を定義する。

ULAgent-SenderはCTC上に常駐し、LogManagerが完成後に`ULAgent`ディレクトリへ移動したログファイルを検出し、送信対象判定、送信状態管理、DataServer Windows上でWindows Serviceとして動作するULAgent-Receiverへのファイルストリーミング送信を行う。

送信状態はSQLiteでログファイル単位に管理し、通信断、ULAgent-Sender再起動、CTC再起動時にも欠損および重複を抑止する。

ULAgent-ReceiverはDataServer Windows上のSelf-contained EXEとして配置され、TLSありgRPC Bidirectional StreamingでULAgent-Senderからログファイルを受信する。
ULsightおよびULsight用DBはDataServer上のUbuntu VMに配置される。
ULAgent-Receiverは、受信したログファイルをDataServer Windows側に保持し、ファイル確定後にUbuntu VM上のInfluxDB/PostgreSQLへ登録する。

ULAgent-Senderは、ファイル内容を解析しない。入力設定に従って転送metadataを作成し、binary chunkとして内容を変更せずに送信する。

送信対象ログファイル

ログ種別	`log_type`	データ	出力元	ContentFormat	ファイル拡張子	CTC相対パス	データ出力頻度	ファイル単位
時系列ログ	`SAMPLING`	Sampling Data	EDA	`CSV`	`.csv`	`SamplingLog\<Chamber>\yyyyMMdd_HHmm.csv`	100ms周期	デフォルト10分ごとに1ファイル（設定変更可）
イベントログ	`EVENT`	Event Data	GEM	`JSON_LINES`	`.evt`	`EventLog\<接続先>\yyyyMMdd_HH.evt`	都度	1時間ごとに1ファイル
プロセスログ (ProcessJob)	`PROCESS`	Process Data	GEM	`JSON`	`.log`	`ProcessLog\yyyyMMdd_HHmmss.log`	都度	プロセス実行ごとに1ファイル
プロセスログ (Wafer)	`PROCESS`	Process Data	GEM	`JSON`	`.wfr`	`ProcessLog\Wafer\yyyyMMdd_HHmmss_<WaferId>.wfr`	都度	ウェーハ処理ごとに1

ログ種別	log_type	データ	出力元	ContentFormat	ファイル拡張子	CTC相対パス	データ出力頻度	ファイル単位
								ファイル

Sampling Dataは100ms周期で出力されるが、ULAgent-Senderの送信対象は100msごとの個別データではなく、LogManagerにより一定時間単位でファイル化されたULAgentディレクトリ配下のログファイルである。

EVENTログは1行1 JSONオブジェクトのJSON Lines形式で出力され、ContentFormatはJSON_LINESである。
PROCESSログはProcessJobとWaferの2種に分類される。

用語定義

用語	内容
ULAgent	ULAgent-SenderおよびULAgent-Receiverから構成されるログ連携システム全体
ULAgent-Sender	CTC上で動作し、LogManagerが完成後にULAgentディレクトリへ移動したログファイルをULAgent-Receiverへ送信するコンポーネント
ULAgent-Receiver	DataServer Windows上で動作し、ULAgent-Senderから送信されたログファイルを受信してULsight用DBへ登録するコンポーネント
DataServer	ULAgent-Receiverが動作するWindowsサーバー。ULsight用のUbuntu VMをホストする
ULsight VM	DataServer上で動作するLinux、具体的にはUbuntu VM。ULsightおよびULsight用DBが配置される
Sender送信管理DB	ULAgent-Senderが送信状態を管理するSQLite
Receiver受信管理DB	ULAgent-Receiverが受信状態を管理するSQLite
ULsight用DB	ULsightが利用するデータ保存先の総称。SAMPLINGはInfluxDB、EVENT/PROCESSはPostgreSQLへ保存する
ULsight用DB登録	ULAgent-Receiverが確定済みログファイルを読み込み、log_typeおよびcontent_formatに応じてInfluxDBまたはPostgreSQLへ登録する処理
ContentFormat	ファイル内容の形式を示す。CSV、JSON、JSON_LINESの3種
ProcessLogType	PROCESSログの種別。PROCESS_JOBおよびWAFERの2種
ルートディレクトリ	CTC配置においてBINの親ディレクトリ。すべての相対パスはこのルートを基準に解決する
Status API	ULAgent-Senderが提供するlocalhostのみのRead-only HTTP API。/health、/api/status、/api/transfers

---

設計方針

- ULAgent-Senderは、ULAgentを構成する送信側コンポーネントである。
- ULAgent-SenderはCTC上のWindows Serviceとして動作する。
- ULAgent-ReceiverはDataServer Windows上のWindows Serviceとして動作する。
- ULsightおよびULsight用DBはDataServer上のUbuntu VMに配置される。
- ULAgent-Senderは、DataServer Windows上で公開されるULAgent-ReceiverのTLS gRPCエンドポイントへ接続する。
- 4種類のログ入力（SAMPLING、EVENT、PROCESS/ProcessJob、PROCESS/Wafer）を識別し、ログファイル単位で送信状態を管理する。
- Senderはファイル内容を解析しない。入力設定に従って転送metadataを作成し、binary chunkとして内容を変更せずに送信する。
- 送信状態はSQLiteに保持し、ULAgent-Sender停止後も復旧可能にする。
- ULAgent-Receiver側の受信完了応答を確認してから送信済みに更新する。
- 欠損防止を優先し、未送信・送信失敗ファイルは再送可能にする。
- 重複防止のため、ファイル識別子file_idをSHA-256ハッシュで生成し管理する。
- ULAgent-Receiver側にもfile_idによる冪等性を求める。
- データ連携方式はTLSありgRPC Bidirectional StreamingによるFile Streamingとする。
- ULAgent-SenderとULAgent-Receiver間の通信ではTLSを必須とする。
- mTLS、クライアント証明書、アプリケーション認証は使用しない。
- 閉域網前提とする。
- 送信対象ファイルの判別方式は、LogManagerがULAgentディレクトリへ移動済みのファイルを対象とする方式とする。LogManagerはファイル完成後にULAgentディレクトリへ移動するため、Senderでは追加の完成判定を行わない。
- ULAgent-Senderは.NET 8.0のSelf-contained EXEとして配布し、Windows Serviceで管理する。

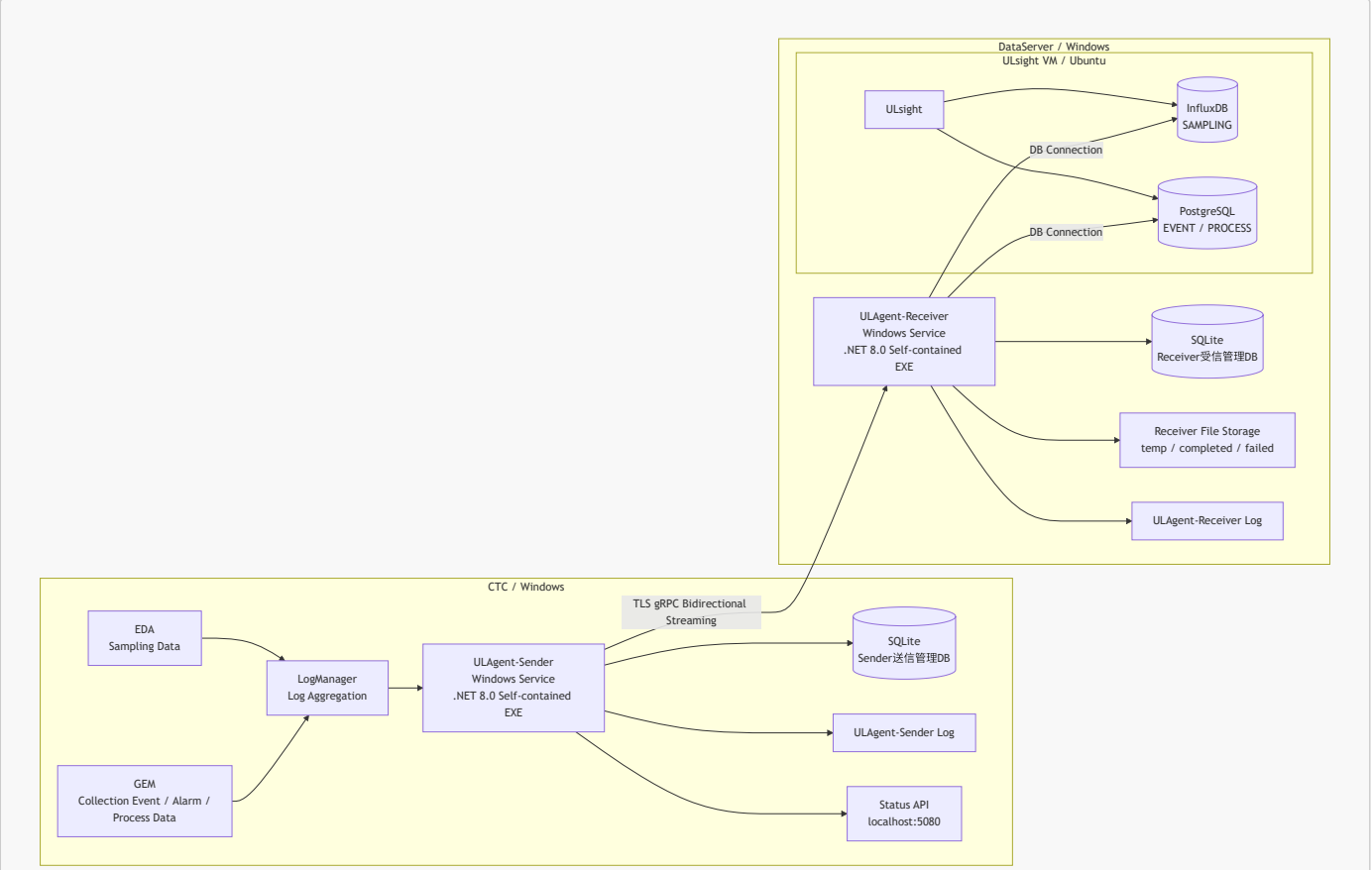
- CTC配置ではBINの親ディレクトリをルートディレクトリとし、すべての相対パスはこのルートを基準に解決する。
- 設定値は外部設定ファイルappsettings.jsonで管理し、現地環境ごとに変更可能にする。
- CTCおよび装置動作への影響を最小化する。
- ネットワーク負荷、ディスクI/O、CPU、メモリ使用量を抑制する。
- 初期実装では同時送信数を1とし、複数ログファイルの並行送信は行わない。
- RETRY_WAIT中のファイルが存在しても、送信キュー全体は停止しない。
- RETRY_WAITのnext_retry_at到達後、対象ファイルをPENDINGへ戻し、queued_atを更新して通常キューへ再投入する。
- 送信対象ファイルは、PENDING状態のファイルからqueued_at昇順で取得する。
- ULAgentディレクトリ内の送信済みログファイルは、Receiverからaccepted=trueを受信しSenderのSENT commitが完了した後に削除する。
- 送信失敗情報はSQLiteに加え、運用確認用の.logファイルにも出力する。
- ULAgent-SenderのSENTは、ULAgent-Receiverがファイル受信・検証・確定を完了した状態を示す。
- ULsight用DB登録完了は、ULAgent-SenderのSENT判定には含めない。
- ULAgent-Senderはlocalhostのみで待受するRead-only Status APIを提供する。POST、PUT、DELETEは提供しない。
- ファイル名のバリデーションとしてLogManagerの正式ファイル名ルールをSenderが検証する。不正なファイル名のファイルはSKIPPEDとして記録する。

技術スタック

カテゴリ	技術
言語	C#
フレームワーク	.NET 8.0
実行形式	Self-contained EXE
実行基盤	Windows Service
OS	Windows 10 / Windows 11（TBD）
通信	TLSありgRPC Bidirectional StreamingによるFile Streaming
gRPC	Grpc.Net.Client
状態管理DB	SQLite (Microsoft.Data.Sqlite)
ファイル保存	LogManager ULAgentディレクトリ配下のログファイル
ログ	Microsoft.Extensions.Logging + Serilog
設定	appsettings.json
データ形式	CSV / JSON / JSON Lines
接続先Receiver配置	DataServer Windows
接続先Receiver実行方式	Self-contained EXE / Windows Service
ULsight配置	DataServer上のUbuntu VM
ULsight用DB	Ubuntu VM上のInfluxDB / PostgreSQL
Status API	ASP.NET Core Minimal API（localhost Read-only）

Architecture

システム構成



CTC配置ディレクトリ構成

CTC配置では、`BIN`の親ディレクトリをルートディレクトリとする。 すべての相対パスはこのルートディレクトリを基準に解決し、Windows Serviceのcurrent working directoryは使用しない。 絶対パスとして設定された値はそのまま使用する。

```
<CTC root>\
├─ BIN\
│  └─ ULAgent.Sender.exe
├─ Config\
│  └─ ULAgent.Sender\
│     └─ appsettings.json
├─ Log\
│  └─ ULAgent.Sender\
│     └─ sender_state.db
└─ ULAgent\
   ├─ EventLog\
   │  └─ <接続先>\
   │     └─ yyyyMMdd_HH.evt
   ├─ SamplingLog\
   │  └─ <Chamber>\
   │     └─ yyyyMMdd_HHmm.csv
   └─ ProcessLog\
      ├─ yyyyMMdd_HHmmss.log
      └─ Wafer\
         └─ yyyyMMdd_HHmmss_<WaferId>.wfr
```

開発環境では、実行ファイルが配置されたディレクトリ名が`BIN`でない場合、`AppContext.BaseDirectory`をテストルートとして使用する。そのため、`dotnet run`およびテストの実行時に本番CTCディレクトリを作成する必要はない。開発PC上で本番配置を模擬する場合は、環境変数`ULAGENT_ROOT`をテストルートに設定する。

CTC内には、EDA、GEM、LogManager、ULAgent-Sender、Sender送信管理DB、ULAgent-Senderログ、Status APIが存在する。

EDAおよびGEMから出力されたログファイルは、LogManagerによりCTC内で集約される。 LogManagerは、ファイル完成後に以降更新されない状態になってから`ULAgent`ディレクトリへ移動する。そのため、Senderでは追加の完成判定を行わない。

ULAgent-Senderは、ULAgentディレクトリ配下のログファイルを送信対象として検出する。 送信対象は各Input設定のLogType、ContentFormat、SearchPatternごとに判定し、ファイル名のバリデーション（LogManagerの正式ファイル名ルール）も実施する。 検出したファイルはSender送信管理DBであるSQLiteに登録し、送信状態を管理する。

ULAgent-Senderは、送信対象ログファイルをチャンク単位に分割し、TLSありgRPC Bidirectional StreamingでDataServer Windows上のULAgent-Receiverへ送信する。 ULAgent-ReceiverからTransferResult accepted=trueを受信した場合、ULAgent-SenderはSenderのSENT commitが完了した後、ULAgentディレクトリ内の送信済みログファイルを削除する。

DataServer側では、ULAgent-ReceiverがWindows Serviceとして動作する。 ULAgent-Receiverは、DataServer Windows上でTLS gRPCエンドポイントを公開する。 ULAgent-Senderは、このエンドポイントへ接続する。

ULsightおよびULsight用DBは、DataServer上のUbuntu VMに配置される。 ULAgent-Receiverは、受信したログファイルをDataServer Windows側のローカルディスクに保持し、ファイル確定後にUbuntu VM上のInfluxDB/PostgreSQLへ登録する。

対象ログファイル設計

時系列ログ

項目	内容
ログ種別	SAMPLING
SEMI関連ワード	Sampling Data
出力元	EDA
集約先	LogManager
データ出力頻度	100ms周期
ファイル単位	デフォルト10分ごとに1ファイル（設定変更可）
ContentFormat	CSV
対象拡張子	.csv
CTC相対パス	SamplingLog\<Chamber>\yyyyMMdd_HHmm.csv
ファイル名ルール	yyyyMMdd_HHmm.csv
SearchPattern	*.csv
SearchSubdirectories	true
送信先	ULAgent-Receiver
最終登録先	InfluxDB

イベントログ

項目	内容
ログ種別	EVENT
SEMI関連ワード	Collection Event / Alarm / Equipment Event
出力元	GEM
集約先	LogManager
データ出力頻度	都度
ファイル単位	1時間ごとに1ファイル
ContentFormat	JSON_LINES
対象拡張子	.evt
CTC相対パス	EventLog\<接続先>\yyyyMMdd_HH.evt
ファイル名ルール	yyyyMMdd_HH.evt
SearchPattern	*.evt
SearchSubdirectories	true
送信先	ULAgent-Receiver

項目	内容
最終登録先	PostgreSQL

EVENTログは1行1 JSONオブジェクトのJSON Lines形式で記録される。ReceiverはJSON逐行解析によりbatch登録する。

プロセスログ (ProcessJob)

項目	内容
ログ種別	`PROCESS`
ProcessLogType	`PROCESS_JOB`
SEMI関連ワード	Process Data / Process Program / Recipe
出力元	GEM
集約先	LogManager
データ出力頻度	都度
ファイル単位	プロセス実行ごとに1ファイル
ContentFormat	`JSON`
対象拡張子	`.log`
CTC相対パス	`ProcessLog\yyyyMMdd_HHmmss.log`
ファイル名ルール	`yyyyMMdd_HHmmss.log`
SearchPattern	`*.log`
SearchSubdirectories	`false`
送信先	ULAgent-Receiver
最終登録先	PostgreSQL

プロセスログ (Wafer)

項目	内容
ログ種別	`PROCESS`
ProcessLogType	`WAFER`
SEMI関連ワード	Process Data / Wafer
出力元	GEM
集約先	LogManager
データ出力頻度	都度
ファイル単位	ウェーハ処理ごとに1ファイル
ContentFormat	`JSON`
対象拡張子	`.wfr`
CTC相対パス	`ProcessLog\Wafer\yyyyMMdd_HHmmss_<WaferId>.wfr`
ファイル名ルール	`yyyyMMdd_HHmmss_<WaferId>.wfr`
SearchPattern	`*.wfr`
SearchSubdirectories	`false`
RelativePathPrefix	`Wafer`
送信先	ULAgent-Receiver
最終登録先	PostgreSQL

Senderの責務

ULAgent-Senderは、CTC上でLogManagerが`ULAgent`ディレクトリへ移動したログファイルを検出し、ULAgent-Receiverへ安全に送信するコンポーネントである。

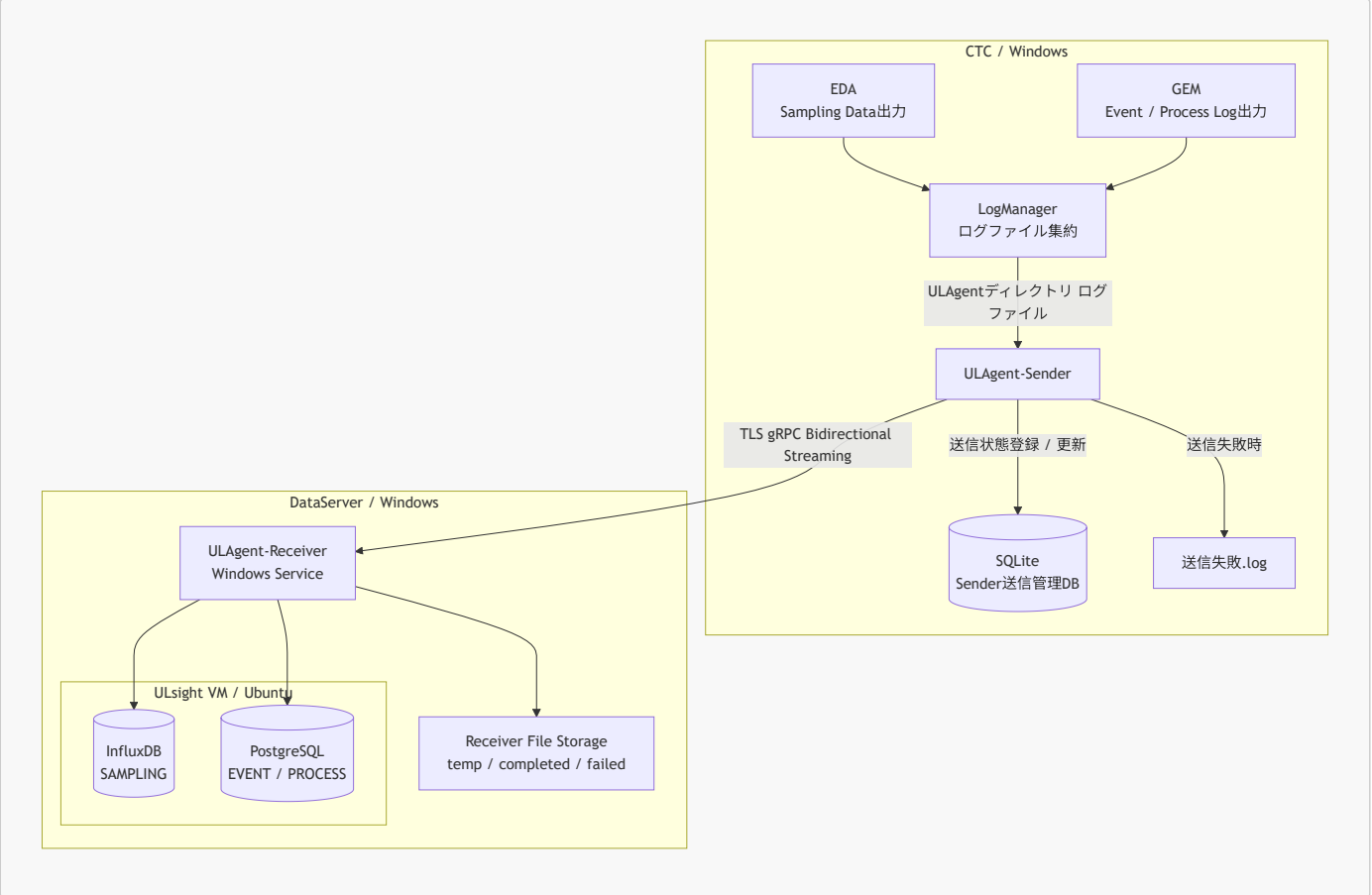
責務	内容
ログファイル検出	各Input設定に基づきLogManagerの`ULAgent`ディレクトリ配下のログファイルを検出する
送信対象判定	`LogType`、`ContentFormat`、`SearchPattern`に基づいて送信対象を判定する
ファイル名バリデーション	LogManagerの正式ファイル名ルールを検証し、不正なファイルは`SKIPPED`とする
送信状態管理	SQLiteで`file_id`単位に送信状態を管理する
重複送信防止	SHA-256ハッシュによる`file_id`で同一ファイルの重複登録・重複送信を抑止する
チャンク分割	ログファイルをチャンク単位に分割する
gRPC送信	TLSありgRPC Bidirectional StreamingでULAgent-Receiverへ送信する
Ack確認	`StartFileAck`、`ChunkAck`、`TransferResult`を確認する
再送制御	Ack未完了チャンクから再送する
送信完了管理	`TransferResult accepted=true`受信後にSQLiteを`SENT`へ更新する
送信済みファイル削除	`SENT` commit完了後、`ULAgent`ディレクトリ内の送信済みログファイルを削除する
削除安全性確認	削除対象パスがInput設定のディレクトリ内であることを検証する
送信失敗ログ出力	送信失敗時、SQLiteに加えて`.log`ファイルへ失敗情報を出力する
障害復旧	ULAgent-Sender再起動後、未送信・送信中断ファイルを復旧する
リソース配慮	CTCや装置動作に影響しないようI/O、CPU、ネットワーク負荷を抑える
ログ出力	ファイル検出、送信開始、送信完了、エラーをULAgent-Senderログへ出力する
Status API提供	localhost Read-only APIで`/health`、`/api/status`、`/api/transfers`を提供する

責務外

項目	理由
EDA/GEMによるログ生成	EDA/GEM側の責務
LogManagerによるログ集約・ファイル完成判定	LogManager側の責務
ログファイル内容の解析	Senderはbinary chunkとして送信し、ファイル内容を解析しない
ULAgent-Receiver側の受信状態管理	ULAgent-Receiverの責務
ULsight用DB登録	ULAgent-Receiverの責務
InfluxDB/PostgreSQL登録結果の確認	ULAgent-Senderの`SENT`判定には含めない
ログ本文の業務的解釈	ULsight用DB登録側の責務

データフロー

Sender中心の全体フロー

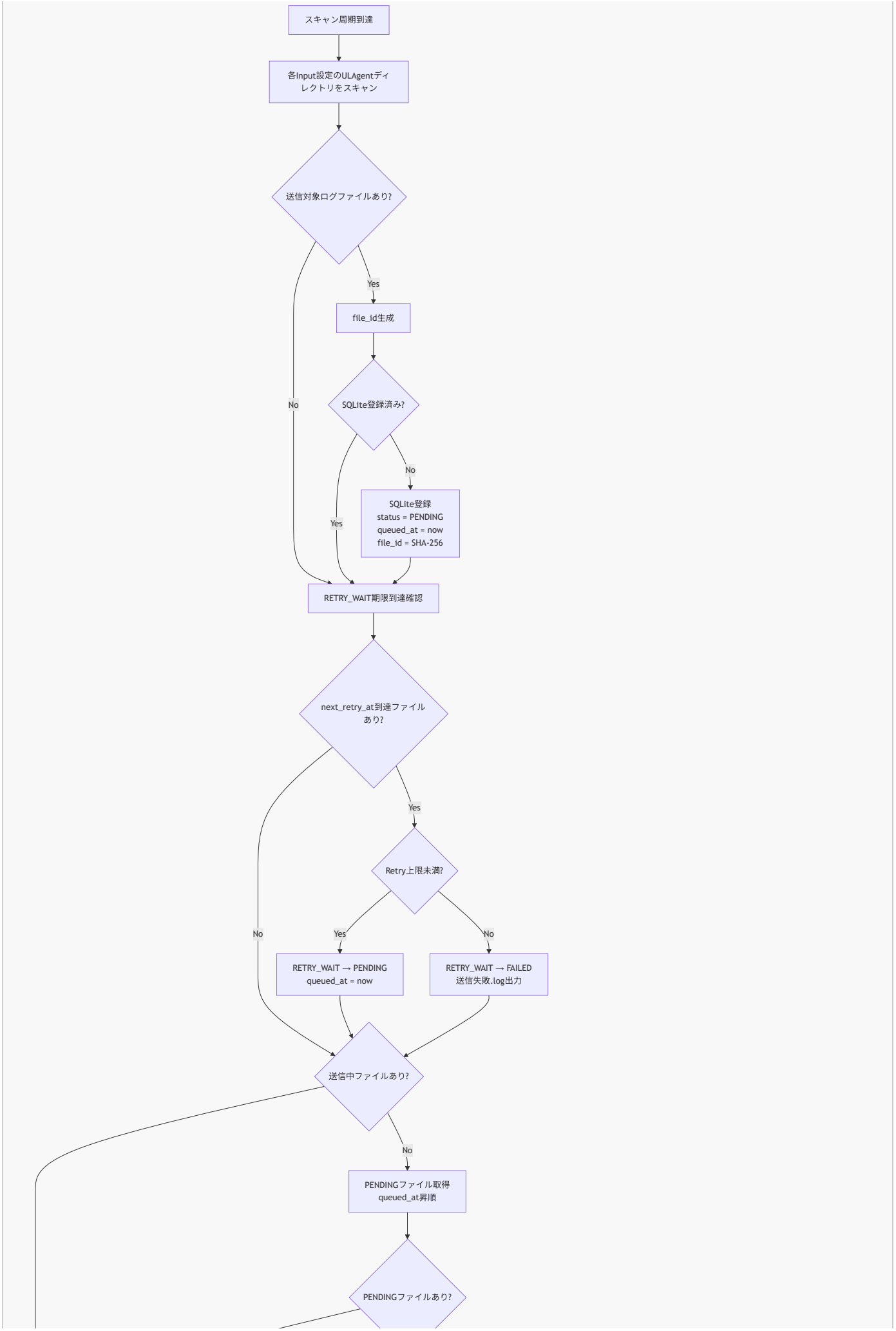


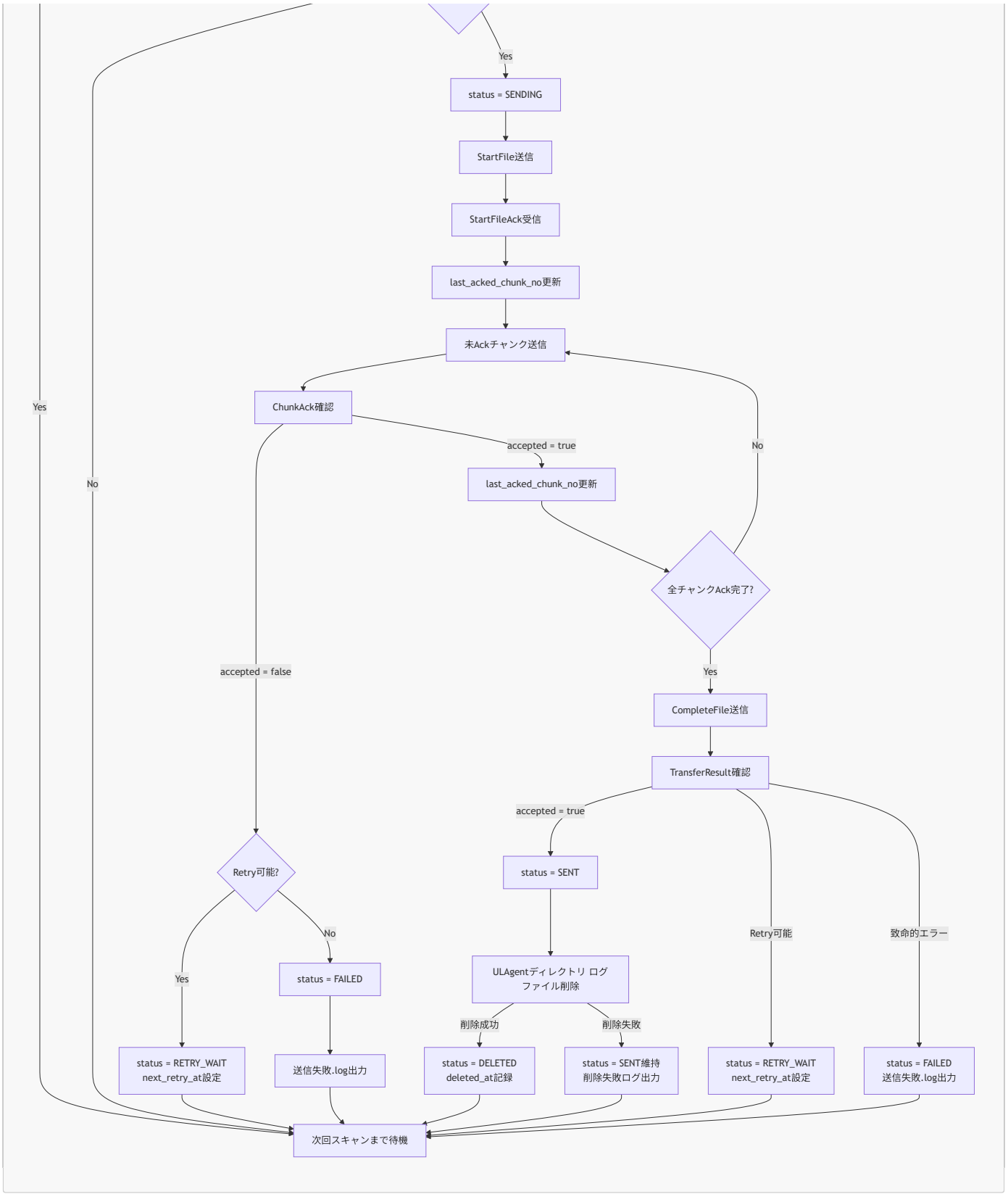
ULAgent-Senderは、CTC内のULAgentディレクトリに配置されたログファイルを検出し、SQLiteで送信状態を管理しながらDataServer Windows上のULAgent-Receiverへ送信する。

ULAgent-Receiverは、受信したログファイルをDataServer Windows側に保持し、ファイル受信・検証・確定完了時点でTransferResult accepted=trueを返却する。

その後、ULAgent-Receiverはlog_typeおよびfile_formatに応じて、Ubuntu VM上のInfluxDB/PostgreSQLへ登録する。 ULsight用DB登録結果はULAgent-Senderへ通知しない。

Sender内部処理フロー


スキャン周期到達
各Input設定のULAgentディレクトリをスキャン
送信対象ログファイルあり?
Yes
No
file_id生成
SQLite登録済み?
No
Yes
SQLite登録
status = PENDING
queued_at = now
file_id = SHA-256
RETRY_WAIT期限到達確認
next_retry_at到達ファイルあり?
Yes
No
Retry上限未満?
Yes
No
RETRY_WAIT → PENDING
queued_at = now
RETRY_WAIT → FAILED
送信失敗.log出力
送信中ファイルあり?
No
PENDINGファイル取得
queued_at昇順
PENDINGファイルあり?



送信処理中でもスキャン周期に到達した場合、ULAgent-Senderは各Input設定のULAgentディレクトリをスキャンする。 ただし、MaxConcurrentTransfers = 1のため、送信中ファイルが存在する場合は新規送信を開始しない。

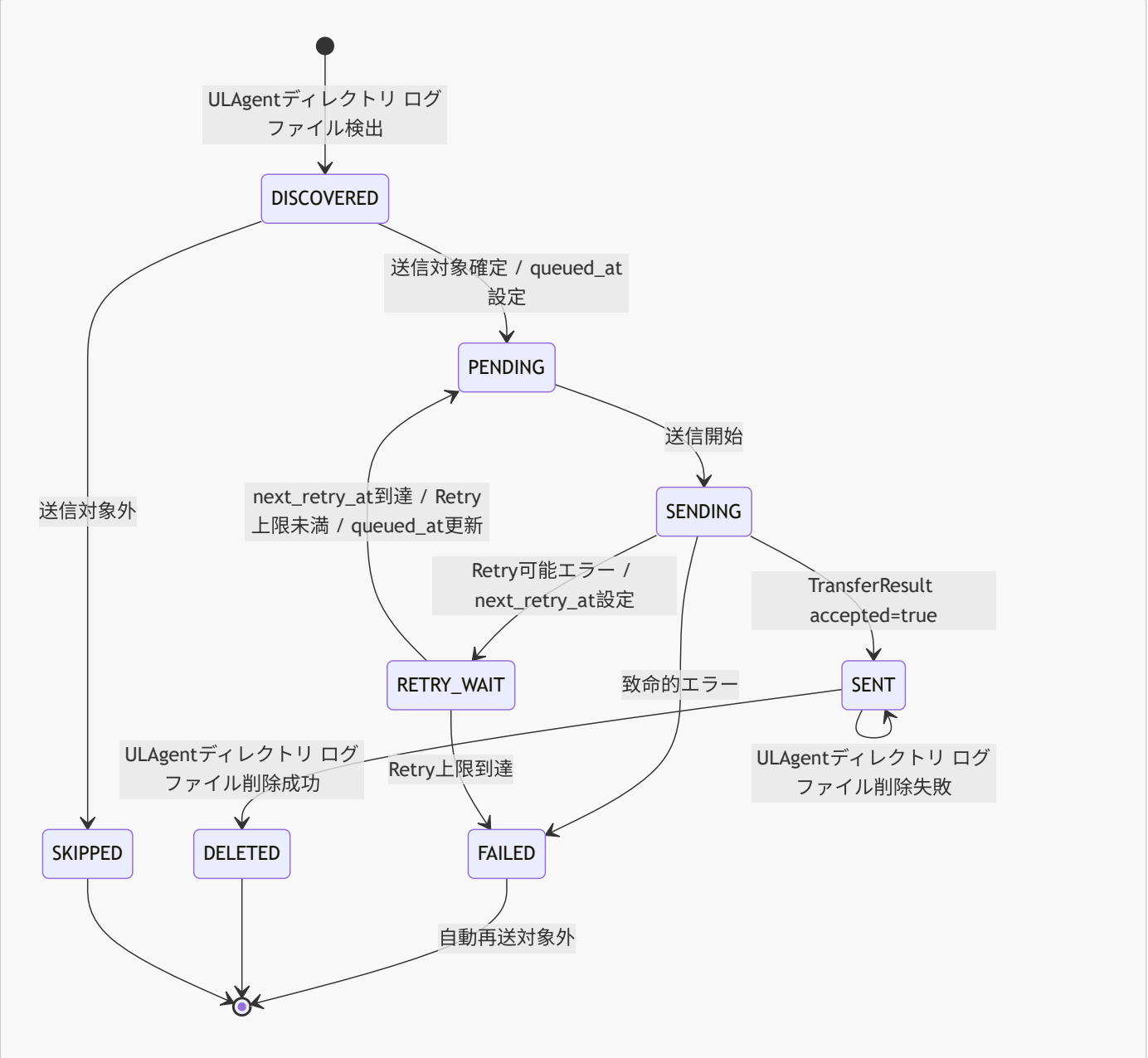
新規に送信対象ファイルを検出した場合、ULAgent-SenderはSQLiteへstatus = PENDINGとして登録し、queued_atに現在時刻を設定する。 file_idはSHA-256ハッシュにより生成する（詳細は「file_id生成」節を参照）。 ファイル名バリデーションに合格しないファイルはstatus = SKIPPEDとして記録する。

RETRY_WAIT状態のファイルは、next_retry_atに到達するまで送信対象外とする。 next_retry_atに到達し、Retry上限に達していない場合、ULAgent-Senderは対象ファイルをPENDINGへ戻し、queued_atを現在時刻で更新する。

Retry上限に達した場合、ULAgent-Senderは対象ファイルをFAILEDへ更新し、送信失敗.logへ出力する。 FAILED状態のファイルは自動再送対象外とする。

送信対象ファイルは、PENDING状態のファイルからqueued_at昇順で取得する。 これにより、RETRY_WAIT中のファイルが存在しても送信キュー全体を停止せず、送信可能なファイルから順次送信する。

Sender送信状態遷移フロー



SENTは、ULAgent-Receiverがファイル受信・検証・確定を完了した状態を示す。
DELETEDは、ULAgentディレクトリ内の送信済みログファイル削除が完了した状態を示す。 FAILEDは、Retry上限到達または致命的エラーにより自動再送対象外となった状態を示す。 ULsight用DB登録完了は、ULAgent-SenderのSENT判定には含めない。

ステータス別処理方針

ステータス	意味	ULAgent-Senderの処理
DISCOVERED	ファイル検出済み	ULAgentディレクトリ配下で検出したログファイルについて、送信対象判定を行う
PENDING	未送信 / 送信待ち	送信キューへ投入し、queued_at昇順で送信開始可能になるまで待機する
SENDING	送信中	ULAgent-ReceiverへTLSありgRPC Bidirectional Streamingでチャンク送信する
RETRY_WAIT	再試行待機中	next_retry_at到達までは送信対象外とする。Retry上限未満の場合はPENDINGへ自動復帰し、queued_atを更新して通常キューに合流する
SENT	送信済み	全チャンクAck完了後、CompleteFileの成功応答を受信済み。ULAgentディレクトリ内の送信済みログファイル削除を実行する

ステータス	意味	ULAgent-Senderの処理
DELETED	削除済み	ULAgentディレクトリ内の送信済みログファイル削除済み。通常の送信処理は終了
FAILED	送信失敗	Retry上限到達または致命的エラー。自動再送対象外とし、送信失敗.logへ失敗情報を出力する。ログファイルは削除せず保持する
SKIPPED	送信対象外	対象外ファイルとして処理終了

送信対象ファイル判別

送信対象ファイルの判別方式は、LogManagerがULAgentディレクトリへ移動済みのファイルを対象とする方式とする。

LogManagerはファイル完成後、以降更新されない状態になってからULAgentディレクトリへ移動する。そのため、Senderでは追加の完成判定を行わない。ULAgentディレクトリへ移動された後はSenderがファイルを管理し、Receiverからaccepted=trueを受信してSenderのSENT commitが完了した後、Senderが送信元ファイルを削除する。

ディレクトリ構成

```
<CTC root>\ULAgent\EventLog\<接続先>\yyyyMMdd_HH.evt
<CTC root>\ULAgent\SamplingLog\<Chamber>\yyyyMMdd_HHmm.csv
<CTC root>\ULAgent\ProcessLog\yyyyMMdd_HHmmss.log
<CTC root>\ULAgent\ProcessLog\Wafer\yyyyMMdd_HHmmss_<WaferId>.wfr
```

項目	内容
判別方式	LogManager ULAgentディレクトリ移動方式
送信対象	各Input設定のSearchPatternに合致し、ファイル名バリデーションに合格するファイル
送信対象外	SearchPatternに合致しないファイル、ファイル名ルール不正のファイル
対象ログ	SAMPLING / EVENT / PROCESS（ProcessJob / Wafer）
完了判定	LogManagerによりULAgentディレクトリへ移動されたことをもって書き込み完了とみなす。Senderでは追加の完成判定を行わない

ファイル名バリデーション

Senderは、LogManagerの正式ファイル名ルールをファイル検出時に検証する。不正なファイル名のファイルはSKIPPEDとして記録する。

log_type	ProcessLogType	ファイル名ルール	例
SAMPLING	-	yyyyMMdd_HHmm.csv	20260714_1200.csv
EVENT	-	yyyyMMdd_HH.evt	20260714_12.evt
PROCESS	PROCESS_JOB	yyyyMMdd_HHmmss.log	20260714_120000.log
PROCESS	WAFER	yyyyMMdd_HHmmss_<WaferId>.wfr	20260714_120000_WF001.wfr

ディレクトリ構造バリデーション

Senderは、ファイルのディレクトリ構造も検証する。

log_type	必須構造
SAMPLING	SamplingLog\<Chamber>\ファイル名（サブディレクトリ直下に1階層）
EVENT	EventLog\<接続先>\ファイル名（サブディレクトリ直下に1階層）
PROCESS (ProcessJob)	ProcessLog\ファイル名（直下）
PROCESS (Wafer)	ProcessLog\Wafer\ファイル名（Waferサブディレクトリ直下）

判定方針

- ULAgent-Senderは各Input設定のDirectoryをスキャン対象とする。
- SearchSubdirectoriesがtrueの場合、サブディレクトリも再帰的にスキャンする。
- SearchPatternでファイル拡張子のフィルタを行う。
- ファイル名フォーマットおよびディレクトリ構造のバリデーションを実行する。
- バリデーション不合格のファイルはSKIPPED状態としてSQLiteに記録する。

ストリーミング

ストリーミング方式

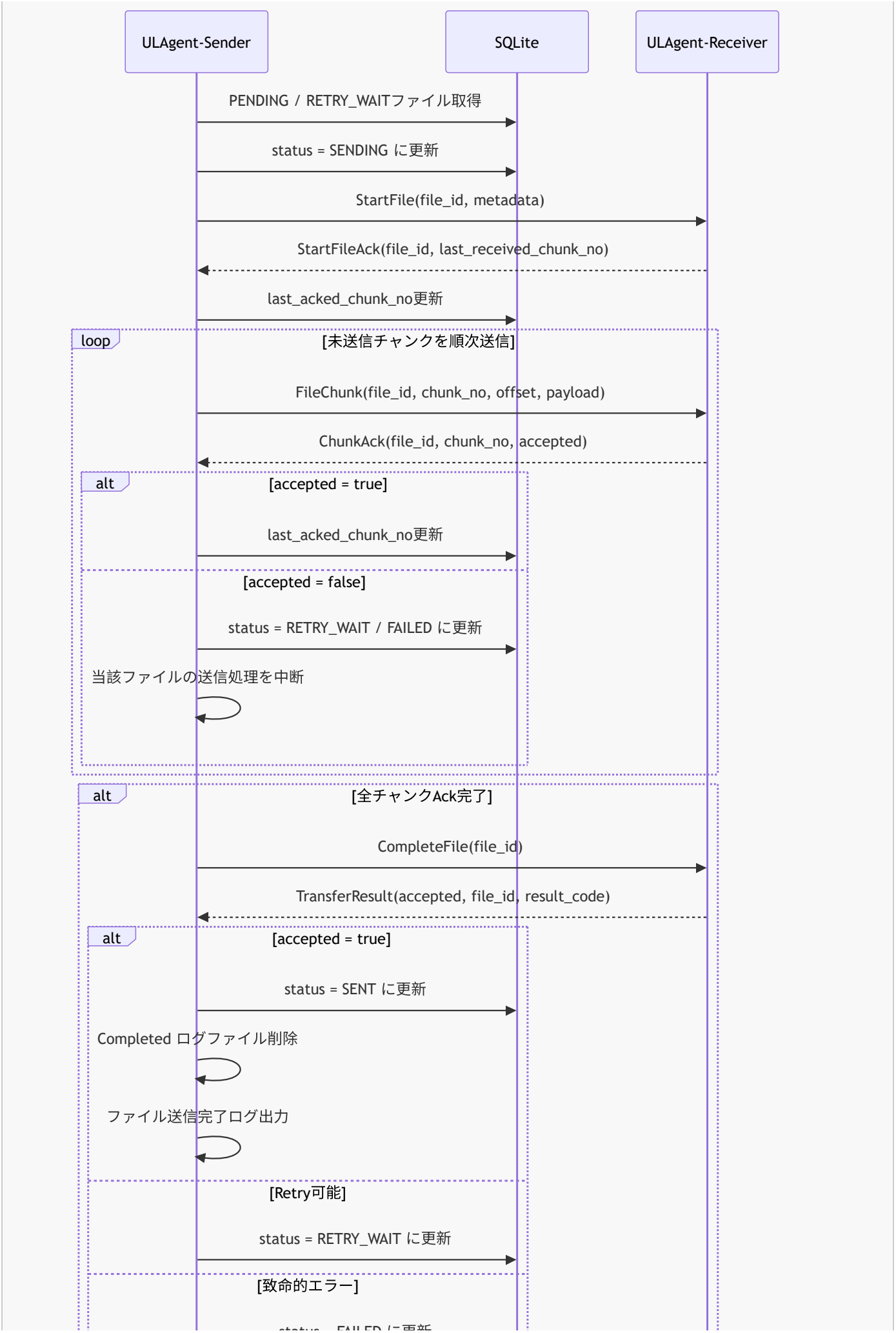
ULAgent-Receiverへのデータ連携は、TLSありのgRPC File Streamingを使用する。
ULAgent-SenderとULAgent-Receiver間の通信ではTLSを必須とする。

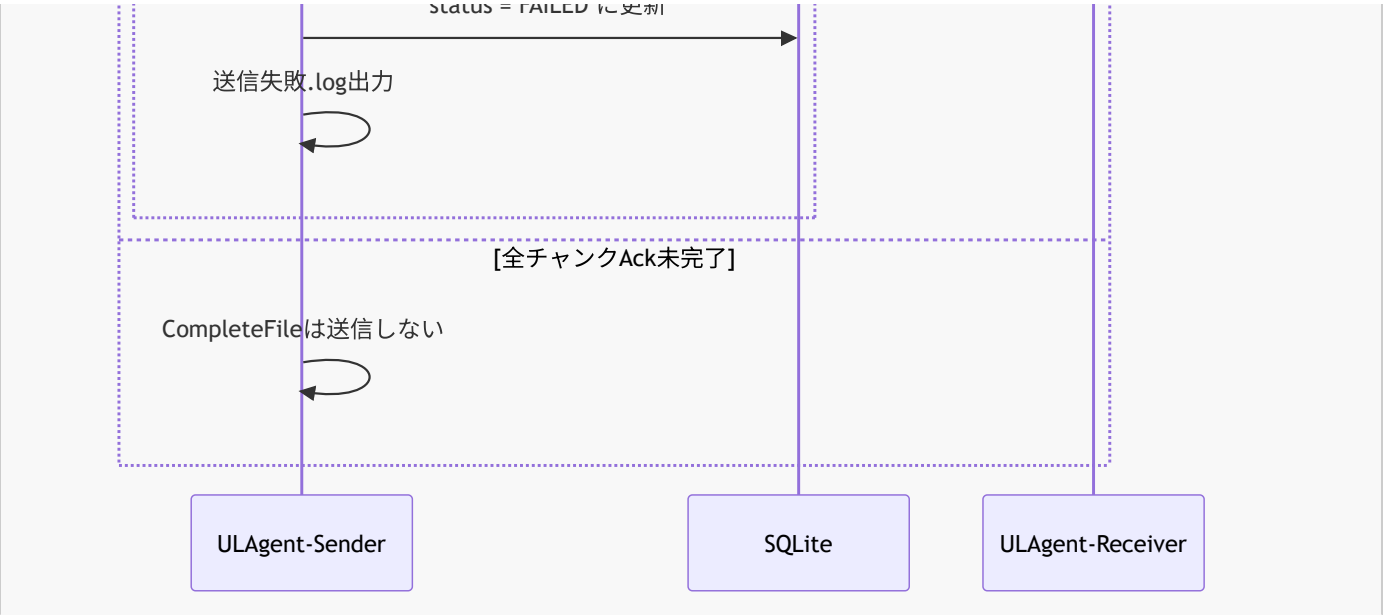
項目	仕様
通信方式	gRPC File Streaming
通信暗号化	TLS必須
接続先形式	`https://<DataServerHost>:<ReceiverPort>`
接続先	DataServer Windows上のULAgent-Receiver
Receiver実行方式	Windows Service / Self-contained EXE
送信単位	ファイルをチャンク分割して送信
ストリーム種別	Bidirectional Streaming
チャンクサイズ	256KB暫定。64KB〜1MBで調整
再送単位	チャンク単位
Ack	チャンク単位Ackを使用する
ファイル完了応答	`CompleteFile`に対するULAgent-Receiverの成功応答で確認する
サーバー証明書検証	必須
クライアント証明書	使用しない
mTLS	使用しない
認証	使用しない。閉域網前提

ULAgent-Senderはログファイルをチャンクに分割して送信する。
チャンク送信ごとにULAgent-ReceiverからAckを受信し、Ack未完了のチャンクがある場合は当該チャンクから再送する。
全チャンクAck完了後、`CompleteFile`を送信し、ULAgent-Receiverから`TransferResult accepted=true`を受信した時点でファイル送信完了とする。

ULAgent-Senderの`SENT`は、DataServer Windows上のULAgent-Receiverがファイル受信・検証・確定を完了したことを意味する。
Ubuntu VM上のInfluxDB/PostgreSQLへのULsight用DB登録完了は、ULAgent-Senderの`SENT`判定には含めない。

ストリーミングフロー


ULAgent-Sender
SQLite
ULAgent-Receiver
PENDING / RETRY_WAITファイル取得
status = SENDING に更新
StartFile(file_id, metadata)
StartFileAck(file_id, last_received_chunk_no)
last_acked_chunk_no更新
loop
[未送信チャンクを順次送信]
FileChunk(file_id, chunk_no, offset, payload)
ChunkAck(file_id, chunk_no, accepted)
alt
[accepted = true]
last_acked_chunk_no更新
[accepted = false]
status = RETRY_WAIT / FAILED に更新
当該ファイルの送信処理を中断
alt
[全チャンクAck完了]
CompleteFile(file_id)
TransferResult(accepted, file_id, result_code)
alt
[accepted = true]
status = SENT に更新
Completed ログファイル削除
ファイル送信完了ログ出力
[Retry可能]
status = RETRY_WAIT に更新
[致命的エラー]



送信メタデータ

項目	内容
file_id	ファイル識別子（SHA-256ハッシュ、64文字hex）
equipment_id	Equipment識別子
ctc_id	CTC識別子
log_type	SAMPLING / EVENT / PROCESS
source_system	EDA / GEM
file_format	CSV / JSON / JSON_LINES
file_name	ファイル名
file_size	ファイルサイズ
last_write_time	最終更新時刻
chunk_size	チャンクサイズ
total_chunks	総チャンク数
relative_path	CTC ULAgentディレクトリ内の相対パス
process_log_type	PROCESSログの種別。PROCESS_JOB / WAFER / 空文字列

source_systemは、SAMPLINGの場合はEDA、EVENTおよびPROCESSの場合はGEMとする。
file_formatは、SAMPLINGの場合はCSV、EVENTの場合はJSON_LINES、PROCESSの場合はJSONとする。
relative_pathは、Receiverがファイルを保存する際のディレクトリ構造の維持に使用される。
process_log_typeは、PROCESSの場合のみPROCESS_JOBまたはWAFERを設定する。それ以外は空文字列とする。

チャンクメタデータ

項目	内容
file_id	ファイル識別子
chunk_no	チャンク番号。0始まり
offset	ファイル先頭からのバイト位置
chunk_size	チャンクサイズ
payload	チャンクデータ本体
is_last	最終チャンクかどうか

Ack仕様

StartFileAck

項目	内容
file_id	ファイル識別子
accepted	受信開始可否
last_received_chunk_no	ULAgent-Receiverが受信済みの最終チャンク番号
result_code	結果コード
message	詳細メッセージ

ChunkAck

項目	内容
file_id	ファイル識別子
chunk_no	Ack対象のチャンク番号
accepted	チャンク受信可否
result_code	結果コード
message	詳細メッセージ

TransferResult

項目	内容
file_id	ファイル識別子
accepted	ファイル受信完了可否
result_code	結果コード
message	詳細メッセージ

ULAgent-Senderは、`TransferResult.accepted = true`の場合のみ、SQLiteの`status`を`SENT`へ更新する。

---

gRPC API設計

gRPC APIの詳細は、ULAgent-Receiver設計書の`gRPC API設計`に従う。
ULAgent-Senderは、共通の`.proto`定義から生成したgRPC Clientを使用する。

状態管理 / Data Models

SQLiteデータモデル

`file_transfer_status`

ログファイル単位で送信状態を管理する。

項目	型	値 / 例	内容
id	INTEGER	1	内部ID
file_id	TEXT	A1B2C3...（64文字hex）	ファイル識別子（SHA-256ハッシュ）
equipment_id	TEXT	EQP001	Equipment識別子
ctc_id	TEXT	CTC001	CTC識別子
log_type	TEXT	SAMPLING / EVENT / PROCESS	ログ種別
source_system	TEXT	EDA / GEM	出力元
file_path	TEXT	C:\...\ULAgent\SamplingLog\CH1\20260714_1200.csv	ファイルフルパス
file_format	TEXT	CSV / JSON / JSON_LINES	ContentFormat
file_name	TEXT	20260714_1200.csv	ファイル名

項目	型	値 / 例	内容
file_size	INTEGER	1048576	ファイルサイズ
last_write_time	TEXT	2026-07-14T12:00:00+09:00	最終更新時刻
relative_path	TEXT	CH1/20260714_1200.csv	ULAgentディレクトリ内の相対パス
process_log_type	TEXT	PROCESS_JOB / WAFER / 空文字列	PROCESSログの種別
status	TEXT	PENDING	送信状態
status_reason	TEXT	SAMPLING filename must be yyyyMMdd_HHmm.csv.	SKIPPED時の理由
retry_count	INTEGER	0	再試行回数
queued_at	TEXT	2026-07-14T12:00:00+09:00	送信キュー投入日時
next_retry_at	TEXT	2026-07-14T12:05:00+09:00	次回再試行日時
chunk_size	INTEGER	262144	チャンクサイズ
total_chunks	INTEGER	4	総チャンク数
last_acked_chunk_no	INTEGER	2	最後にAck済みのチャンク番号。未Ackの場合は-1
last_error_code	TEXT	NETWORK_TIMEOUT	最終エラーコード
last_error_message	TEXT	Timeout occurred	最終エラーメッセージ
created_at	TEXT	2026-07-14T12:00:01+09:00	登録日時
updated_at	TEXT	2026-07-14T12:00:10+09:00	更新日時
sent_at	TEXT	2026-07-14T12:00:15+09:00	送信完了日時
deleted_at	TEXT	2026-07-14T12:10:00+09:00	送信済みファイルログ削除日時
delete_error_message	TEXT	Access denied	削除失敗時のエラーメッセージ

status値

値	内容
DISCOVERED	ファイル検出済み
PENDING	未送信 / 送信待ち
SENDING	送信中
RETRY_WAIT	再試行待機中
SENT	送信済み
FAILED	送信失敗
SKIPPED	送信対象外
DELETED	送信済みログファイル削除済み

log_type値

値	内容
SAMPLING	EDA由来のSampling Data
EVENT	GEM由来のCollection Event / Alarm等のイベントログ
PROCESS	GEM由来のProcess Data（ProcessJob / Wafer）

content_format値

値	内容
CSV	SAMPLING用。CSV形式
JSON	PROCESS用。単一JSONオブジェクト形式

値	内容
JSON_LINES	EVENT用。1行1 JSONオブジェクトのJSON Lines形式

process_log_type値

値	内容
PROCESS_JOB	プロセス実行ごとのジョブログ
WAFER	ウェーハ処理ごとのウェーハログ
空文字列	PROCESS以外のlog_typeの場合

file_id生成

file_idは、ファイルの一意識別のためSHA-256ハッシュにより生成する。
ハッシュ入力は以下の要素を|で連結した文字列とする。

```
SHA-256( ctcId | logType | filePath(大文字化) | fileSize | lastWriteTimeUtc.Ticks )
```

要素	説明
ctcId	CTC識別子
logType	ログ種別の文字列表現（SAMPLING、EVENT、PROCESS）
filePath	ファイルフルパスを大文字化した値
fileSize	ファイルサイズ（バイト数、数値文字列）
lastWriteTimeUtc.Ticks	最終更新日時のUTC Ticks（数値文字列）

生成結果は64文字の16進数文字列（大文字hex）となる。
Receiverはfile_idが64文字hexであることをバリデーションする。

DDL案

```
CREATE TABLE IF NOT EXISTS file_transfer_status (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_id TEXT NOT NULL UNIQUE,
    equipment_id TEXT NOT NULL,
    ctc_id TEXT NOT NULL,
    log_type TEXT NOT NULL,
    source_system TEXT NOT NULL,
    file_path TEXT NOT NULL,
    file_format TEXT NOT NULL,
    file_name TEXT NOT NULL,
    file_size INTEGER NOT NULL,
    last_write_time TEXT NOT NULL,
    relative_path TEXT NOT NULL DEFAULT '',
    process_log_type TEXT NOT NULL DEFAULT '',
    status TEXT NOT NULL,
    status_reason TEXT,
    retry_count INTEGER NOT NULL DEFAULT 0,
    queued_at TEXT,
    next_retry_at TEXT,
    chunk_size INTEGER,
    total_chunks INTEGER,
    last_acked_chunk_no INTEGER NOT NULL DEFAULT -1,
    last_error_code TEXT,
    last_error_message TEXT,
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    sent_at TEXT,
    deleted_at TEXT,
    delete_error_message TEXT
);

CREATE INDEX IF NOT EXISTS idx_file_transfer_status_status
```

```
ON file_transfer_status(status);

CREATE INDEX IF NOT EXISTS idx_file_transfer_status_log_type
ON file_transfer_status(log_type);

CREATE INDEX IF NOT EXISTS idx_file_transfer_status_queued_at
ON file_transfer_status(queued_at);

CREATE INDEX IF NOT EXISTS idx_file_transfer_status_next_retry_at
ON file_transfer_status(next_retry_at);
```

SQLite履歴データ保持

Sender送信管理DBの完了済みデータ（SENT、DELETED、FAILED、SKIPPED）は、初期パラメータとして**30日間**保持する。
30日を経過した完了済みレコードは自動削除の対象とする。

処理中または再試行対象のデータ（PENDING、SENDING、RETRY_WAIT）は、自動削除対象外とする。

項目	値
保持期間	30日
対象ステータス	SENT、DELETED、FAILED、SKIPPED
自動削除対象外	PENDING、SENDING、RETRY_WAIT
削除基準日時	updated_atから30日経過

ファイル操作

欠損防止

- ファイル検出後、送信前にSQLiteへ登録する。
- 送信完了はULAgent-Receiverからの受信完了応答をもって確定する。
- SENTへ更新する前にULAgent-Senderが停止した場合は、次回起動時に再送する。
- CTC再起動後もSQLiteを基準に未送信ファイルを復旧する。
- ULAgent-Senderは各Input設定のULAgentディレクトリ配下のログファイルのみを送信対象とする。
- 送信未完了、送信失敗、再試行待機中のファイルは削除しない。
- 送信開始前にファイルサイズと最終更新日時を検証し、変更があった場合は送信を中止する。

送信済みファイル削除

ULAgent-Senderは、ULAgentディレクトリ内のログファイルについて、送信完了後に削除を行う。

削除条件

条件	内容
送信完了	ULAgent-ReceiverからTransferResult accepted=trueを受信していること
状態更新完了	SQLiteのstatusがSENTへcommit完了していること
パス安全性確認	削除対象パスがInput設定のディレクトリ内であることを検証していること

削除しない条件

状態	理由
PENDING	未送信のため
SENDING	送信中のため
RETRY_WAIT	再送対象のため
FAILED	調査または手動再送の可能性があるため
SKIPPED	送信対象外のため

削除失敗時の扱い

ファイル削除に失敗した場合、ULAgent-Senderは送信状態を`SENT`のままとし、削除失敗をULAgent-Senderログへ出力する。
`delete_error_message`に失敗理由を記録する。
削除失敗ファイルの再削除方法は運用で検討する。

削除パス安全性確認

ULAgent-Senderは、削除対象パスが設定済みInput設定のいずれかのディレクトリ配下であることを検証してから削除を行う。 設定されたディレクトリ外のパスは削除を拒否する。

Error Handling

TLS / 証明書エラー

エラー分類	例	対応
TLS接続エラー	TLSハンドシェイク失敗	`RETRY_WAIT`へ更新し、ULAgent-Senderログへ記録。Retry上限超過時は`FAILED`
サーバー証明書エラー	証明書期限切れ、信頼チェーン不正	送信を停止し、対象ファイルは`RETRY_WAIT`または`FAILED`へ更新。ULAgent-SenderログおよびWindows Event Logへ出力
ホスト名不一致	証明書CN/SANと接続先ホスト名が不一致	接続失敗。`RETRY_WAIT`または`FAILED`へ更新し、設定または証明書を見直す

エラーハンドリング戦略

エラー分類	例	対応
ファイルアクセスエラー	ファイルロック、権限不足	再試行。継続失敗時は`FAILED`または`RETRY_WAIT`
通信エラー	ULAgent-Receiver未接続、Timeout	`RETRY_WAIT`へ更新し、指数バックオフで再試行
Receiverエラー	5xx相当	`RETRY_WAIT`へ更新し、指数バックオフで再試行
Receiver拒否	形式不正、メタデータ不正	`FAILED`に更新しULAgent-Senderログ出力
SQLiteエラー	DBロック、書き込み失敗	短周期リトライ。復旧不可時はULAgent-Senderログ出力
ULAgent-Sender異常終了	プロセス停止、OS再起動	次回起動時に`SENDING`を`PENDING`へ戻す
ディスク容量不足	DB/ULAgent-Senderログ書き込み不可	ULAgent-Senderログ、Windows Event Log等に出力。運用対応
Completedファイル削除失敗	ファイルロック、権限不足	`SENT`状態は維持し、削除失敗ログを出力

Retry制御

ULAgent-Senderは、送信失敗時に即時再送を行わず、Retry可能なエラーの場合は対象ファイルを`RETRY_WAIT`へ更新する。
`RETRY_WAIT`更新時には、`next_retry_at`を設定する。

`RETRY_WAIT`状態のファイルは、`next_retry_at`に到達するまで送信対象外とする。
`next_retry_at`に到達し、Retry上限に達していない場合、ULAgent-Senderは対象ファイルを`PENDING`へ戻し、`queued_at`を現在時刻で更新して通常キューへ再投入する。

Retry上限に達した場合、ULAgent-Senderは対象ファイルを`FAILED`へ更新する。
`FAILED`状態のファイルは自動再送対象外とし、Completedディレクトリ内に保持する。
また、送信失敗情報を送信失敗`.log`へ出力する。

`RETRY_WAIT`中のファイルが存在しても、送信キュー全体は停止しない。
他に`PENDING`状態のファイルが存在する場合、ULAgent-Senderは当該ファイルを送信する。

送信失敗ログ管理

ULAgent-Senderは、SQLiteによる送信状態管理に加え、送信失敗情報を別途`.log`ファイルへ出力する。

SQLiteは送信制御のための正式な状態管理として使用する。
`.log`ファイルは、運用者が送信失敗内容をすぐ確認するための補助情報として使用する。

出力先

```
<CTC root>\Log\ULAgent.Sender\transfer_failed.log
```

出力タイミング

タイミング	内容
Retry上限超過	`FAILED`遷移時に出力
Receiver拒否	メタデータ不正、形式不正等で`FAILED`になった場合に出力
ファイルアクセス継続失敗	ファイルロック、権限不足等で`FAILED`になった場合に出力
TLS/通信エラー継続失敗	Retry上限超過時に出力

リソース制限

CTCは装置由来のログが集約されるPCであり、装置側の動作に影響を与えてはならない。

項目	方針
同時送信数	初期値は`1`
ファイル読み込み	チャンク単位で読み込み、一括ロードしない
DB更新	`last_acked_chunk_no`の更新はチャンクAck単位を基本とする。ただし負荷状況に応じて一定間隔での更新も検討する
フォルダ監視	定期スキャンを基本とする。FileSystemWatcher併用は将来検討
スキャン周期	初期値は`60000ms`、つまり1分間隔とする
ネットワーク	帯域制限を設定可能にする
ディスクI/O	読み込み帯域制限を設定可能にする
プロセス優先度	低優先度設定を可能にする

Runtime Configuration

`appsettings.json`例

```
{
  "Sender": {
    "EquipmentId": "TBD",
    "CtcId": "CTC001",
    "ScanIntervalMilliseconds": 60000,
    "MaxConcurrentTransfers": 1,
    "StatusApiPort": 5080,
    "ReceiverEndpoint": "https://<DataServerHost>:5001",
    "ChunkSizeBytes": 262144,
    "RetryLimit": 5,
    "RetryBaseDelaySeconds": 10,
    "TimeoutSeconds": 300,
    "DatabasePath": "Log\\ULAgent.Sender\\sender_state.db",
    "CompletedHistoryRetentionDays": 30,
    "Inputs": [
      {
        "LogType": "Sampling",
        "ContentFormat": "Csv",
        "Directory": "ULAgent\\SamplingLog",
        "SearchPattern": "*.csv",
        "SearchSubdirectories": true
      },
      {
        "LogType": "Event",
        "ContentFormat": "JsonLines",
        "Directory": "ULAgent\\EventLog",
        "SearchPattern": "*.evt",
        "SearchSubdirectories": true
      },
      {
        "LogType": "Process",
        "ProcessLogType": "ProcessJob",
        "ContentFormat": "Json",
        "Directory": "ULAgent\\ProcessLog",
```

```
        "SearchPattern": "*.log"
      },
      {
        "LogType": "Process",
        "ProcessLogType": "Wafer",
        "ContentFormat": "Json",
        "Directory": "ULAgent\\ProcessLog\\Wafer",
        "RelativePathPrefix": "Wafer",
        "SearchPattern": "*.wfr"
      }
    ]
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

設定ファイルの配置先は`<CTC root>\Config\ULAgent.Sender\appsettings.json`とする。
`DatabasePath`および各Input設定の`Directory`は、相対パスの場合ルートディレクトリからの相対パスとして解決される。絶対パスの場合はそのまま使用する。
`ReceiverEndpoint`には、DataServer Windows上で公開されるULAgent-ReceiverのTLS gRPCエンドポイントを指定する。

設定項目

セクション	項目	内容
`Sender`	`EquipmentId`	Equipment識別子
`Sender`	`CtcId`	CTC識別子
`Sender`	`ScanIntervalMilliseconds`	ULAgentディレクトリのスキャン間隔（最小1000ms）
`Sender`	`MaxConcurrentTransfers`	同時送信数（1〜16）
`Sender`	`StatusApiPort`	Status APIの待受ポート（デフォルト5080）
`Sender`	`ReceiverEndpoint`	DataServer Windows上で公開されるULAgent-ReceiverのTLS gRPCエンドポイント
`Sender`	`ChunkSizeBytes`	送信チャンクサイズ（64KB〜1MB）
`Sender`	`RetryLimit`	最大再試行回数（0〜100）
`Sender`	`RetryBaseDelaySeconds`	再試行ベース待機秒数（指数バックオフで増加）
`Sender`	`TimeoutSeconds`	送信タイムアウト（1〜86400秒）
`Sender`	`DatabasePath`	Sender送信管理DBのSQLiteファイルパス（相対パスの場合ルートディレクトリ基準）
`Sender`	`CompletedHistoryRetentionDays`	SQLite完了済み履歴の保持日数（初期値30日）
`Sender.Inputs[]`	`LogType`	ログ種別。`Sampling` / `Event` / `Process`
`Sender.Inputs[]`	`ContentFormat`	ファイル内容形式。`Csv` / `Json` / `JsonLines`
`Sender.Inputs[]`	`Directory`	スキャン対象ディレクトリ（相対パスの場合ルートディレクトリ基準）
`Sender.Inputs[]`	`SearchPattern`	ファイル検索パターン（例: `*.csv`、`*.evt`、`*.log`、`*.wfr`）
`Sender.Inputs[]`	`SearchSubdirectories`	サブディレクトリを再帰スキャンするか（デフォルト`false`）
`Sender.Inputs[]`	`RelativePathPrefix`	relative_pathに付与するプレフィックス（Wafer用）
`Sender.Inputs[]`	`ProcessLogType`	PROCESSの場合のみ必須。`ProcessJob` / `Wafer`

バリデーション

- 全LogType（`Sampling`、`Event`、`Process`）について最低1つのInput設定が必要。
- `Process`については`ProcessJob`および`Wafer`の両方のInput設定が必要。
- 同一`LogType`/`ProcessLogType`の組み合わせの重複は不可。
- `ContentFormat`は`LogType`との対応が正しいこと（`Sampling`=`Csv`、`Event`=`JsonLines`、`Process`=`Json`）。
- `Process`以外の`LogType`に`ProcessLogType`を設定した場合はエラー。

運用設計

起動契機

起動契機	内容
OS起動時	Windows Serviceの起動種別`Automatic`により自動起動
手動開始	Services MMC、PowerShell、`sc.exe`等により開始
異常終了後	Windows Service Recoveryにより自動再起動

Windows Service

項目	内容
サービス名	`ULAgent.Sender`
表示名	`ULAgent Sender`
起動種別	Automatic
実行ユーザー	LocalServiceまたは専用ユーザー（TBD）
死活監視	Windows Service監視
異常終了時	Windows Service Recoveryで自動再起動
配置先	`<CTC root>\BIN\ULAgent.Sender.exe`
設定ファイル	`<CTC root>\Config\ULAgent.Sender\appsettings.json`
DBファイル	`<CTC root>\Log\ULAgent.Sender\sender_state.db`

起動時処理

1. ルートディレクトリを決定する（`BIN`親ディレクトリ、または`ULAGENT_ROOT`環境変数）。
2. `appsettings.json`を読み込む（`<root>\Config\ULAgent.Sender\appsettings.json`）。
3. 設定バリデーションを実行する（Input設定の整合性確認）。
4. SQLite DBを初期化する。
5. 前回 `SENDING` だったファイルを異常中断扱いとし、再送対象として `PENDING` へ戻す。
6. `RETRY_WAIT` 状態のファイルについて `next_retry_at` 到達有無とRetry上限を確認する。
7. `next_retry_at` 到達済みかつRetry上限未満のファイルを `PENDING` へ戻し、`queued_at = now` に更新する。
8. Retry上限到達済みのファイルを `FAILED` に更新し、送信失敗 `.log` へ出力する。
9. 各Input設定のULAgentディレクトリをスキャンする。
10. 未登録ファイルをSQLiteへ登録する。
11. `PENDING`ファイルの送信を開始する。
12. Status APIの待受を開始する（`http://127.0.0.1:<StatusApiPort>`）。

停止時処理

1. 新規送信開始を停止する。
2. 送信中ファイルを可能な範囲で完了させる。
3. 中断した場合は`PENDING`へ戻す。
4. ULAgent-Senderログへ停止情報を出力する。

Logging

ULAgent-Senderは、ファイル検出、送信対象判定、送信状態遷移、ULAgent-Receiver応答、エラー、リソース状態をログ出力する。
ログは障害調査、欠損・重複確認、運用監視を目的とし、ログファイル本文などの機密情報は出力しない。

ログ出力方式

項目	方針
ログ基盤	`Microsoft.Extensions.Logging`
ファイル出力	`Serilog`
出力先	`<CTC root>\Log\ULAgent.Sender\`

項目	方針
ローテーション	装置側運用方針に合わせるためTBD
保持期間	装置側運用方針に合わせるためTBD
タイムゾーン	UTC（PLC側でUTCに統一されていることを前提とする）
重大エラー	Windows Event Logにも出力
通常ログレベル	`Information`
詳細調査時	`Debug`

出力対象

分類	主な出力内容	レベル
Sender起動停止	起動、停止、バージョン、設定読込結果	`Information`
設定チェック	監視ディレクトリ、Receiver接続先、設定不正	`Information` / `Error`
ファイル検出	`file_id`、ログ種別、ファイル名、サイズ	`Information`
送信対象判定	`PENDING`、`SKIPPED`の判定理由	`Information` / `Warning`
SQLite状態更新	ステータス変更、retry_count更新	`Information`
送信処理	送信開始、送信完了、送信サイズ、処理時間	`Information`
Receiver応答	ChunkAck、TransferResultの成功、拒否、不正応答	`Information` / `Warning` / `Error`
リトライ	Retry回数、次回Retry時刻、理由	`Warning`
送信失敗	`FAILED`遷移、失敗ログ出力	`Error`
ファイル削除	Completedログファイル削除成功/失敗	`Information` / `Warning`
リソース状態	キュー滞留、ディスク容量不足、帯域制限発動	`Information` / `Warning`

出力禁止情報

- ログファイル本文全体
- Recipe詳細などの機密性が高いProcess条件
- 個人情報
- 過剰なチャンク単位ログ

Status API

ULAgent-Senderは、localhostのみで待受するRead-only HTTP APIを提供する。
POST、PUT、DELETEは提供しない。デフォルトでは`127.0.0.1`のみでlistenする。

エンドポイント

メソッド	パス	内容
`GET`	`/health`	ヘルスチェック。起動日時、稼働時間を返却する
`GET`	`/api/status`	送信状態のサマリー（各ステータスの件数、最終送信成功日時）
`GET`	`/api/transfers?limit=n`	最近の送信レコード一覧（最大500件、デフォルト50件）

レスポンス例

`/health`

```
{
  "status": "healthy",
  "startedAt": "2026-07-14T12:00:00+00:00",
  "uptimeSeconds": 3600
}
```

`/api/status`

```
{
  "pending": 5,
  "sending": 1,
  "retryWait": 2,
  "sent": 100,
  "failed": 0,
  "skipped": 3,
  "lastSuccessfulTransferAt": "2026-07-14T12:10:00+00:00"
}
```

`/api/transfers?limit=n`

```
[
  {
    "fileId": "A1B2C3...",
    "fileName": "20260714_1200.csv",
    "logType": "SAMPLING",
    "status": "SENT",
    "retryCount": 0,
    "lastAcknowledgedChunkNumber": 3,
    "totalChunks": 4,
    "updatedAt": "2026-07-14T12:10:00+00:00",
    "lastErrorCode": null,
    "statusReason": null
  }
]
```

セキュリティ

項目	方針
通信暗号化	ULAgent-SenderとULAgent-Receiver間の通信にはTLSを必須とする
接続方式	TLSありのgRPC接続を使用する
接続先URL	DataServer Windows上で公開されるULAgent-ReceiverのTLS gRPCエンドポイント。形式は`https://<DataServerHost>:<ReceiverPort>`
サーバー認証	ULAgent-SenderはULAgent-Receiverのサーバー証明書を検証する
証明書検証	証明書チェーン、有効期限、ホスト名一致を検証する
クライアント証明書	使用しない
mTLS	使用しない
アプリケーション認証	使用しない。閉域網前提
接続先制限	ULAgent-ReceiverはDataServer Windows上に配置され、閉域網内に公開される前提とする
設定ファイル保護	認証情報は平文保存を許容する。ただしDataServer運用方針に応じて保護する

未定事項・要検討事項

分類	項目	暫定方針
再送	手動再送方法	管理CLIまたはDBステータス更新を検討
Receiver接続	DataServer Windows上のReceiver公開ポート	デフォルト5001
Receiver接続	DataServer Windowsのホスト名 / IP	現地ネットワーク構成に合わせて設定
TLS証明書	Receiverサーバー証明書の発行・配置	DataServer Windows上のReceiver設計に合わせる

分類	項目	暫定方針
ネットワーク	CTCからDataServer Windowsへの到達性	閉域網、FW、ポート開放方針に合わせる
ログ	ログローテーション方式	装置側運用方針に合わせる
ログ	タイムゾーン	UTC統一（確定済み）
改善案	累積Ack、`committed_offset`、SHA-256チェックサム	大容量ファイル実測後に導入判断。現行初期リリースには含めない

受入条件

初期リリース時点の基本的な受入条件は以下とする。
具体的な性能基準値および試験条件は、本設計書では定義しない。

- ULAgent-SenderがWindows Serviceとして起動・停止できること。
- 4種類のログ入力（SAMPLING、EVENT、PROCESS/ProcessJob、PROCESS/Wafer）を識別できること。
- 各Input設定に基づき`ULAgent`ディレクトリ配下のログファイルを送信対象として検出できること。
- ファイル名バリデーションにより不正なファイルを`SKIPPED`として処理できること。
- SQLiteで送信状態を管理できること。
- DataServer Windows上のULAgent-Receiverへファイルストリーミング送信できること。
- TLSありgRPC接続でDataServer Windows上のULAgent-Receiverへ送信できること。
- `ReceiverEndpoint`として`https://<DataServerHost>:<ReceiverPort>`形式の接続先を設定できること。
- 通信失敗時にAck未完了のチャンクから再試行できること。
- ULAgent-Sender再起動後に未送信ファイルを再送できること。
- SHA-256ハッシュによる`file_id`で同一ファイルの重複登録を防止できること。
- ULAgent-Receiverから`TransferResult accepted=true`を受信した場合に`SENT`へ更新できること。
- `SENT`がULAgent-Receiverでのファイル受信・検証・確定完了を意味し、Ubuntu VM上のInfluxDB/PostgreSQLへの登録完了を含まないこと。
- `SENT` commit完了後、`ULAgent`ディレクトリ内の送信済みログファイルを削除できること。
- 送信失敗時に`.log`ファイルへ失敗情報を出力できること。
- CTCおよび装置側の動作に影響する過大な負荷を発生させない設計であること。
- Status API（`/health`、`/api/status`、`/api/transfers`）がlocalhostから応答できること。
- `relative_path`および`process_log_type`を送信メタデータとして正しく設定できること。
- ファイルサイズおよび最終更新日時の変更を検出し、送信を中止できること。

前提・仮定

- ログファイルはCTC上のローカルディスクに出力され、ファイル形式は`log_type`ごとに異なる。
- `SAMPLING`はCSV形式、`EVENT`はJSON Lines形式（`.evt`拡張子）、`PROCESS`はJSON形式（`.log`または`.wfr`拡張子）とする。
- LogManagerは、ファイル完成後に以降更新されない状態になってから`ULAgent`ディレクトリへ移動する。
- Senderでは追加の完成判定を行わない。`ULAgent`ディレクトリへ移動された後はSenderがファイルを管理する。
- ULAgent-Senderは`ULAgent`ディレクトリ配下のログファイルを送信対象とする。
- ULAgent-SenderはCTC上でWindows Serviceとして常駐する。
- ULAgent-SenderはULAgentを構成する送信側コンポーネントである。
- CTC配置では`BIN`の親ディレクトリをルートディレクトリとし、すべての相対パスはこのルートを基準に解決する。
- 開発環境では`AppContext.BaseDirectory`をテストルートとして使用する。本番配置を模擬する場合は環境変数`ULAGENT_ROOT`で指定する。
- DataServer自体はWindowsである。
- ULAgent-ReceiverはDataServer Windows上でWindows Serviceとして動作する。
- ULAgent-ReceiverはSelf-contained EXEとして配置される。
- ULAgent-ReceiverはDataServer Windows上でTLS gRPCエンドポイントを公開する。
- ULAgent-Senderは、DataServer Windows上で公開されるULAgent-ReceiverのTLS gRPCエンドポイントへ接続する。
- ULsightおよびULsight用DBは、DataServer上のUbuntu VMに配置される。
- ULAgent-Receiverは、受信したログファイルをDataServer Windows側に保持する。
- ULAgent-Receiverは、DataServer WindowsからUbuntu VM上のInfluxDB/PostgreSQLへ接続してULsight用DB登録を行う。
- ULAgent-Receiver側は受信完了応答および`file_id`による冪等処理に対応する想定とする。
- ULAgent-Receiver側は、`file_id`単位でチャンク受信状態を管理し、`StartFileAck`で受信済み最終チャンク番号を返却できるものとする。
- ULAgent-Senderの`SENT`は、ULAgent-Receiverがファイル受信・検証・確定を完了した状態を示し、InfluxDB/PostgreSQLへの登録完了は含まない。
- ULAgent-Receiverは、ファイル確定後に`log_type`に応じてInfluxDBまたはPostgreSQLへ登録する。
- ULAgent-Senderは、`SENT` commit完了後に`ULAgent`ディレクトリ内の送信済みログファイルを削除する。
- ULAgent-Receiver側は、同一`file_id`および同一`chunk_no`の再受信を冪等に扱うものとする。
- `file_id`はSHA-256ハッシュ（64文字hex）であり、Receiverは`file_id`のフォーマットをバリデーションする。
- ULAgent-Senderで扱うログファイル名、ファイル更新時刻、送信管理DB上の時刻形式はUTCとする。PLC側でUTCに統一されていることを前提とする。

- 本設計では対象CTCは1台とし、同一CTC内で同名ファイルは存在しない前提とする。
- ULAgent-Senderはファイル内容を解析せず、binary chunkとして送信する。